

Doc 9.3 — `break`, `continue` & Labeled Statements

এই ডকটি **Loops & Iteration** Section-এর শেষ অংশ।

এখানে আমরা শিখব কীভাবে loop-এর normal flow control করা যায় — কখন loop পুরোপুরি বন্ধ করব, কখন শুধু current iteration skip করব, আর nested loop-এর ক্ষেত্রে কীভাবে নির্দিষ্ট outer loop control করা যায়।

1. `break` Statement

`break` ব্যবহার করলে JavaScript সাথে সাথে nearest loop বা switch থেকে বের হয়ে যায়।

Basic Syntax

```
break;
```

Example:

```
for (let i = 1; i <= 10; i++) {  
  if (i === 5) {  
    break;  
  }  
  
  console.log(i);  
}
```

Output:

```
1  
2  
3  
4
```

কারণ `i === 5` হলে `break` execute হয়েছে এবং loop পুরোপুরি বন্ধ হয়ে গেছে।

2. **break** কীভাবে কাজ করে?

Mental model:

```
Loop starts
  ↓
Iteration
  ↓
Condition
  ↓
break?
  └─┬─┘
Yes  No
  ↓    ↓
Exit Continue
```

অর্থ্যাৎ:

```
for (...) {
    if (condition) {
        break;
    }

    // code
}
```

মানে:

"এই condition true হলে আর loop চালানোর দরকার নেই।"

3. **break** with **for** Loop

```
for (let i = 1; i <= 10; i++) {  
  if (i === 6) {  
    break;  
  }  
  
  console.log(i);  
}
```

Output:

```
1  
2  
3  
4  
5
```

6 আসার আগেই loop terminate হয়েছে।

4. **break** with **while**

```
let i = 1;  
  
while (i <= 10) {  
  if (i === 5) {  
    break;  
  }  
  
  console.log(i);  
  
  i++;  
}
```

Output:

```
1
2
3
4
```

5. `break` with `do...while`

```
let i = 1;

do {
  if (i === 4) {
    break;
  }

  console.log(i);

  i++;
} while (i <= 10);
```

Output:

```
1
2
3
```

6. Real-World Example — User Search

ধরো আমাদের কাছে users আছে এবং আমরা নির্দিষ্ট user খুঁজছি।

```
const users = [
  { id: 1, name: "Rahim" },
  { id: 2, name: "Karim" },
  { id: 3, name: "Shariar" },
  { id: 4, name: "Hasan" }
];

let foundUser = null;

for (const user of users) {
  if (user.name === "Shariar") {
    foundUser = user;
    break;
  }
}

console.log(foundUser);
```

এখানে **Shariar** পাওয়ার পর আর অন্য user check করার দরকার নেই।

তাই:

```
break;
```

দিয়ে loop বন্ধ করা হয়েছে।

7. **break** — Early Exit Pattern

এটা programming-এ খুব common pattern।

```
for (const item of items) {
  if (isValid(item)) {
    result = item;
    break;
  }
}
```

Mental model:

যেটা খুঁজছি সেটা পেয়ে গেলে আর **search** করার দরকার নেই।

এতে unnecessary iteration কমে।

8. `continue` Statement

`continue` পুরো loop বন্ধ করে না।

এটি শুধু **current iteration skip** করে এবং পরবর্তী iteration-এ চলে যায়।

Syntax:

```
continue;
```

Example:

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) {  
    continue;  
  }  
  
  console.log(i);  
}
```

Output:

```
1  
2  
4  
5
```

এখানে `3` skip হয়েছে।

কিন্তু loop বন্ধ হয়নি।

9. `break` vs `continue`

এটা খুব ভালোভাবে বুঝতে হবে।

<code>break</code>	<code>continue</code>
পুরো loop বন্ধ করে	শুধু current iteration skip করে
loop থেকে বের হয়ে যায়	next iteration-এ যায়
আর কোনো iteration হয় না	পরের iteration চলতে থাকে
Search শেষ করতে useful	কিছু item skip করতে useful

Example:

```
if (condition) {
  break;
}
```

মানে:

"আর loop চালাব না"

আর:

```
if (condition) {
  continue;
}
```

মানে:

"এই iteration বাদ দাও, পরেরটায় যাও।"

10. `continue` with `for`

```
for (let i = 1; i <= 10; i++) {
  if (i % 2 === 0) {
    continue;
  }

  console.log(i);
}
```

Output:

```
1
3
5
7
9
```

এখানে even numbers skip হয়েছে।

11. `continue` with `while`

এখানে একটু সাবধান হতে হবে।

```
let i = 0;

while (i < 5) {
  i++;

  if (i === 3) {
    continue;
  }

  console.log(i);
}
```

Output:

```
1
2
4
5
```

এখানে `i++` আগে হওয়ায় loop ঠিকমতো এগিয়েছে।

12. `while` Loop-এ `continue` এর Common Mistake

এই code:

```
let i = 0;

while (i < 5) {
  if (i === 3) {
    continue;
  }

  console.log(i);

  i++;
}
```

এখানে সমস্যা আছে।

যখন:

```
i === 3
```

হবে, তখন:

```
continue;
```

execute হবে।

ফলে:

```
i++;
```

execute হবে না।

তাই `i` সবসময় `3` থেকে যাবে।

ফলে infinite loop হবে।

তাই মনে রাখবে

`while` / `do...while` -এ `continue` ব্যবহার করার সময় loop-এর progress যেন বন্ধ না হয়ে যায়।

13. `continue` দিয়ে Invalid Data Skip করা

Real-world API data processing:

```
const users = [
  { name: "Rahim", age: 25 },
  null,
  { name: "Karim", age: 30 },
  { name: "Hasan", age: 22 }
];

for (const user of users) {
  if (!user) {
    continue;
  }

  console.log(user.name);
}
```

Output:

```
Rahim
Karim
Hasan
```

`null` user skip হয়েছে।

14. `continue` — Validation Pattern

```
const numbers = [10, -5, 20, -2, 30];

for (const number of numbers) {
  if (number < 0) {
    continue;
  }

  console.log(number);
}
```

Output:

```
10
  20
  30
```

Mental model:

```
Invalid → skip
Valid   → process
```

15. **break** + **continue** একসাথে

একই loop-এ দুটোই থাকতে পারে।

```
for (let i = 1; i <= 10; i++) {
  if (i === 3) {
    continue;
  }

  if (i === 8) {
    break;
  }

  console.log(i);
}
```

Output:

```
1
2
4
5
6
7
```

Flow:

```
1 → print
2 → print
3 → skip
4 → print
5 → print
6 → print
7 → print
8 → break
```

16. Nested Loops এবং **break**

এখানে খুব important একটা rule আছে।

```
for (let i = 1; i <= 3; i++) {
  for (let j = 1; j <= 3; j++) {
    if (j === 2) {
      break;
    }

    console.log(i, j);
  }
}
```

Output:

```
1 1
2 1
3 1
```

কেন?

কারণ:

```
break;
```

শুধু **nearest loop** থেকে বের হয়।

অর্থাৎ inner loop বন্ধ হয়েছে।

Outer loop চলতে থেকেছে।

17. Nested Loop-এ `continue`

একইভাবে:

```
for (let i = 1; i <= 2; i++) {  
  for (let j = 1; j <= 3; j++) {  
    if (j === 2) {  
      continue;  
    }  
  
    console.log(i, j);  
  }  
}
```

Output:

```
1 1  
1 3  
2 1  
2 3
```

`continue` inner loop-এর current iteration skip করেছে।

18. Inner Loop থেকে Outer Loop Control করা

কখনো আমাদের দরকার হতে পারে:

Inner loop-এর কোনো condition true হলে পুরো outer loop-ও বন্ধ করতে হবে।

Normal `break` দিয়ে সেটা করা যায় না।

কারণ:

```
break;
```

শুধু nearest loop বন্ধ করে।

এই ধরনের ক্ষেত্রে **labeled statement** ব্যবহার করা যায়।

19. Labeled Statements

Label হলো কোনো statement বা loop-এর একটি নাম দেওয়া।

Syntax:

```
labelName:
  statement
```

Example:

```
outerLoop:
  for (let i = 0; i < 3; i++) {
    // ...
  }
```

এখন এই loop-এর label:

```
outerLoop
```

20. Labeled **break**

সবচেয়ে common use:

```
outerLoop:
  for (let i = 1; i <= 3; i++) {
    for (let j = 1; j <= 3; j++) {
      if (i === 2 && j === 2) {
        break outerLoop;
      }

      console.log(i, j);
    }
  }
```

Output:

```
1 1
  1 2
  1 3
  2 1
```

যখন:

```
i === 2 && j === 2
```

হয়:

```
break outerLoop;
```

পুরো `outerLoop` থেকে বের হয়ে যায়।

21. Normal `break` vs Labeled `break`

Normal:

```
break;
```

Nearest loop থেকে বের হয়।

Labeled:

```
break outerLoop;
```

নির্দিষ্ট labeled loop থেকে বের হয়।

Mental model

```
break
  ↓
nearest loop

break outerLoop
  ↓
outerLoop
```

22. Matrix Search Example

DSA-তে labeled break-এর একটা useful example:

```
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

let found = false;

outerLoop:
for (let row = 0; row < matrix.length; row++) {
  for (let col = 0; col < matrix[row].length; col++) {
    if (matrix[row][col] === 6) {
      found = true;
      break outerLoop;
    }
  }
}

console.log(found);
```

যখন **6** পাওয়া যায়:

```
break outerLoop;
```

দিয়ে দুইটা loop-ই শেষ হয়ে যায়।

23. Labeled **continue**

Label শুধু **break**-এর জন্য না।

continue-এর সাথেও label ব্যবহার করা যায়।

Example:

```
outerLoop:
  for (let i = 1; i <= 3; i++) {
    for (let j = 1; j <= 3; j++) {
      if (j === 2) {
        continue outerLoop;
      }

      console.log(i, j);
    }
  }
```

এখানে:

```
continue outerLoop;
```

inner loop-এর current iteration না, বরং **outer loop-এর next iteration** শুরু করবে।

24. Labeled `continue` কীভাবে কাজ করে?

ধরো:

```
outer i = 1
  ↓
  inner j = 1
    ↓
    inner j = 2
      ↓
      continue outerLoop
        ↓
        outer i = 2
```

অর্থাৎ inner loop থেকে সরাসরি outer loop-এর next iteration-এ চলে যাচ্ছে।

25. `break outerLoop` VS `continue outerLoop`

খুব গুরুত্বপূর্ণ পার্থক্য:

break outerLoop

পুরো outer loop শেষ করে।

```
break outerLoop;
```

মানে:

"সব loop শেষ।"

continue outerLoop

Outer loop-এর current iteration শেষ করে next iteration শুরু করে।

```
continue outerLoop;
```

মানে:

"এই outer iteration বাদ দাও, পরের outer iteration-এ যাও।"

26. Labeled **break** — Practical Example

ধরো duplicate pair খুঁজছি:

```
const numbers = [2, 7, 11, 15];
const target = 9;

let found = false;

outerLoop:
for (let i = 0; i < numbers.length; i++) {
  for (let j = i + 1; j < numbers.length; j++) {
    if (numbers[i] + numbers[j] === target) {
      found = true;
      break outerLoop;
    }
  }
}

console.log(found);
```

27. Labeled Statements খুব বেশি ব্যবহার করা উচিত?

সাধারণত না।

Labeled statements valid JavaScript হলেও everyday code-এ খুব বেশি ব্যবহার করা হয় না।

কারণ:

```
break outerLoop;
```

দেখলে nested control flow একটু complex মনে হতে পারে।

অনেক ক্ষেত্রে cleaner solution হতে পারে:

- function থেকে `return`
- helper function
- normal `break`
- flag variable
- array methods
- অন্যভাবে algorithm restructure করা

28. Flag Variable দিয়ে Alternative

Labeled break-এর পরিবর্তে flag ব্যবহার করা যায়।

```
let found = false;

for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (i === 2 && j === 2) {
      found = true;
      break;
    }
  }

  if (found) {
    break;
  }
}
```

এটা কিছু ক্ষেত্রে readable হতে পারে, যদিও nested loop-এর complexity অনুযায়ী labeled break-ও perfectly valid।

29. **break** in **switch**

break শুধু loop-এ না, **switch** -এও ব্যবহার হয়।

```
const day = 2;

switch (day) {
  case 1:
    console.log("Saturday");
    break;

  case 2:
    console.log("Sunday");
    break;

  default:
    console.log("Unknown");
}
```

এখানে **break** case execute করার পর switch থেকে বের করে দেয়।

30. `switch` -এ `break` না দিলে কী হয়?

```
const value = 1;

switch (value) {
  case 1:
    console.log("One");

  case 2:
    console.log("Two");

  case 3:
    console.log("Three");
}
```

Output:

```
One
Two
Three
```

এটাকে বলে **fall-through**।

কারণ `case 1` -এর পরে `break` নেই।

তাই execution পরের cases-এ চলে গেছে।

31. Intentional Fall-Through

কখনো ইচ্ছা করেও multiple cases একসাথে handle করা যায়।

```
const role = "admin";

switch (role) {
  case "admin":
  case "manager":
    console.log("Can manage users");
    break;

  case "user":
    console.log("Can view profile");
    break;
}
```

এখানে:

```
admin
  manager
    ↓
  Can manage users
```

দুটো role একই behavior share করছে।

32. **return** VS **break**

এটাও interview-এ আসতে পারে।

break

শুধু loop/switch থেকে বের হয়।

```
function test() {  
  for (let i = 0; i < 5; i++) {  
    if (i === 2) {  
      break;  
    }  
  
    console.log(i);  
  }  
  
  console.log("Function still running");  
}
```

Output:

```
0  
1  
Function still running
```

return

পুরো function থেকে বের হয়ে যায়।

```
function test() {  
  for (let i = 0; i < 5; i++) {  
    if (i === 2) {  
      return;  
    }  
  
    console.log(i);  
  }  
  
  console.log("Function still running");  
}
```

Output:

```
0  
1
```

কারণ **return** function-ই শেষ করে দিয়েছে।

Mental Model

```
break
→ loop/switch শেষ

return
→ পুরো function শেষ
```

33. `break`, `continue`, `return` একসাথে বুঝি

```
break
↓
current loop থেকে বের হও

continue
↓
current iteration বাদ দাও

return
↓
পুরো function থেকে বের হও
```

34. Nested Loop — খুব গুরুত্বপূর্ণ Rule

এই code:

```
for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    break;
  }

  console.log(i);
}
```

`break` outer loop বন্ধ করবে না।
শুধু inner loop বন্ধ করবে।

কারণ:

break always affects the nearest enclosing loop.

35. `continue` -এর ক্ষেত্রেও একই Rule

```
for (let i = 0; i < 3; i++) {  
  for (let j = 0; j < 3; j++) {  
    continue;  
  }  
  
  console.log(i);  
}
```

`continue` inner loop-এর current iteration skip করবে।

Outer loop-এর ওপর direct effect নেই।

36. Common Mistake — `break` সব Loop বন্ধ করে মনে করা

ভুল ধারণা:

```
break = সব loop বন্ধ
```

সঠিক:

```
break = nearest loop বন্ধ
```

যদি নির্দিষ্ট outer loop বন্ধ করতে হয়:

```
break outerLoop;
```

37. Common Mistake — `continue` দিয়ে Loop বন্ধ করা

```
if (condition) {  
    continue;  
}
```

এটা loop বন্ধ করে না।

এটা শুধু current iteration skip করে।

যদি loop বন্ধ করতে চাও:

```
break;
```

38. Common Mistake — `while` -এ `continue`

এই pattern dangerous:

```
let i = 0;  
  
while (i < 10) {  
    if (i === 5) {  
        continue;  
    }  
  
    i++;  
}
```

`i === 5` হলে `i++` আর execute হবে না।

ফলে:

```
i = 5  
i = 5  
i = 5  
...
```

Infinite loop হতে পারে।

39. Common Mistake — Unnecessary Labels

প্রতিটা nested loop-এ label ব্যবহার করার দরকার নেই।

থারাপ:

```
first:
  for (...) {
    second:
      for (...) {
        third:
          for (...) {
            // complex logic
          }
        }
      }
    }
```

এতে code বুঝতে কঠিন হয়ে যেতে পারে।

প্রয়োজন অনুযায়ী ব্যবহার করো।

40. **break** কখন ব্যবহার করব?

যখন:

- কোনো item পাওয়া গেছে
- search শেষ
- target পাওয়া গেছে
- invalid condition detect হয়েছে
- আর iteration প্রয়োজন নেই
- loop early terminate করতে হবে

Example:

```
for (const user of users) {
  if (user.id === targetId) {
    foundUser = user;
    break;
  }
}
```

41. **continue** কখন ব্যবহার করব?

যখন:

- invalid item skip করতে হবে
- কিছু specific item process করতে চাই না
- শুধু valid data process করতে চাই
- কিছু condition-এর জন্য current iteration বাদ দিতে চাই

Example:

```
for (const user of users) {  
  if (!user.active) {  
    continue;  
  }  
  
  processUser(user);  
}
```

42. **break** + Search Pattern

```
const numbers = [10, 20, 30, 40, 50];  
  
let result = null;  
  
for (const number of numbers) {  
  if (number === 30) {  
    result = number;  
    break;  
  }  
}  
  
console.log(result);
```

Mental model:

Search

↓

Found?

↓

Yes → break

43. **continue** + Filtering Pattern

```
const numbers = [1, -2, 3, -4, 5];

for (const number of numbers) {
  if (number < 0) {
    continue;
  }

  console.log(number);
}
```

Mental model:

```
Invalid → skip
Valid → process
```

44. DSA Pattern — Matrix Search

Nested loop:

```
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

outerLoop:
for (let row = 0; row < matrix.length; row++) {
  for (let col = 0; col < matrix[row].length; col++) {
    if (matrix[row][col] === 8) {
      console.log("Found");
      break outerLoop;
    }
  }
}
```

এখানে target পাওয়া গেলে পুরো search বন্ধ।

45. DSA Pattern — Skip Invalid Values

```
const numbers = [10, -3, 5, -8, 20];

let sum = 0;

for (const number of numbers) {
  if (number < 0) {
    continue;
  }

  sum += number;
}

console.log(sum);
```

Output:

35

কারণ:

```
10 + 5 + 20 = 35
```

Negative values skip হয়েছে।

46. `break` vs Array Methods

অনেক ক্ষেত্রে loop-এর বদলে array methods ব্যবহার করা যায়।

যেমন search:

```
const user = users.find(user => user.id === 5);
```

এখানে manually:

```
for (...) {  
  if (...) {  
    break;  
  }  
}
```

লিখতে হচ্ছে না।

তবে loop জানা খুব important কারণ:

- complex algorithms
- nested loops
- custom control flow
- DSA
- performance-sensitive logic
- early exit

এসব জায়গায় traditional loops দরকার হতে পারে।

47. `forEach()` -এর সাথে `break`

একটা গুরুত্বপূর্ণ বিষয়:

এভাবে করা যায় না:

```
users.forEach(user => {  
  if (user.id === 5) {  
    break;  
  }  
});
```

এটা syntax error হবে।

কারণ `break` কোনো loop-এর body-তে না থেকে callback-এর ভিতরে আছে।

`forEach()` থেকে early exit করতে চাইলে সাধারণত:

```
for...of
```

ব্যবহার করা বেশি convenient।

```
for (const user of users) {  
  if (user.id === 5) {  
    break;  
  }  
}
```

48. Quick Comparison

Feature	কাজ
<code>break</code>	loop/switch সম্পূর্ণ বন্ধ
<code>continue</code>	current iteration skip
<code>break label</code>	নির্দিষ্ট labeled statement/loop থেকে বের হওয়া
<code>continue label</code>	নির্দিষ্ট labeled loop-এর next iteration
<code>return</code>	পুরো function শেষ
<code>switch break</code>	case execution থামানো

49. Output Practice — 1

Output কী হবে?

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) {  
    break;  
  }  
  
  console.log(i);  
}
```

নিজে আগে solve করো।

50. Output Practice — 2

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) {  
    continue;  
  }  
  
  console.log(i);  
}
```

Output কী হবে?

51. Output Practice — 3

```
for (let i = 1; i <= 3; i++) {  
  for (let j = 1; j <= 3; j++) {  
    if (j === 2) {  
      break;  
    }  
  
    console.log(i, j);  
  }  
}
```

থেয়াল করো `break` কোন loop affect করছে।

52. Output Practice — 4

```
let i = 0;

while (i < 5) {
  i++;

  if (i === 3) {
    continue;
  }

  console.log(i);
}
```

Output predict করো।

53. Coding Challenge 1 — Find Number

একটি array থেকে target number খুঁজে বের করো।

```
const numbers = [10, 20, 30, 40, 50];
const target = 30;
```

Requirement:

- loop ব্যবহার করবে
 - target পেলো `break`
 - পাওয়া গেলো `"Found"`
 - না পাওয়া গেলো `"Not Found"`
-

54. Coding Challenge 2 — Skip Negative Numbers

```
const numbers = [10, -5, 20, -3, 30];
```

`continue` ব্যবহার করে শুধু positive numbers print করো।

Expected:

```
10
20
30
```

55. Coding Challenge 3 — First Even Number

```
const numbers = [1, 3, 7, 8, 10, 12];
```

প্রথম even number পাওয়ার পর আর loop চালাবে না।

Hint:

```
if (number % 2 === 0) {
  // ...
  break;
}
```

Expected:

```
8
```

56. Coding Challenge 4 — Matrix Search

এই matrix:

```
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];
```

8 খুঁজে বের করো।

Requirement:

- nested loop
- target পাওয়া গেলে পুরো search stop
- labeled `break` ব্যবহার করতে পারো

57. Interview Questions

Q1. `break` কী করে?

Loop বা switch-এর execution immediately terminate করে।

Q2. `continue` কী করে?

Current iteration skip করে next iteration-এ যায়।

Q3. `break` কি সব nested loops বন্ধ করে?

না।

শুধু nearest enclosing loop বন্ধ করে।

Q4. Outer loop বন্ধ করতে কী করা যায়?

Labeled `break`:

```
break outerLoop;
```

Q5. `break` আর `continue` এর difference?

```
break
→ পুরো loop বন্ধ

continue
→ current iteration skip
```

Q6. `while` loop-এ `continue` কেন dangerous হতে পারে?

কারণ loop-এর update statement skip হয়ে গেলে condition-এর value পরিবর্তন নাও হতে পারে, যার ফলে infinite loop হতে পারে।

Q7. `break` কি `switch`-এর ভিতরে ব্যবহার করা যায়?

হ্যাঁ।

`switch` case-এর fall-through বন্ধ করতে সাধারণত `break` ব্যবহার করা হয়।

Q8. `return` আর `break` এর difference?

```
break
→ loop/switch থেকে বের হয়

return
→ পুরো function থেকে বের হয়
```

Q9. Labeled `continue` কী করে?

নির্দিষ্ট labeled outer loop-এর next iteration-এ চলে যায়।

Q10. `forEach()` -এর callback-এর ভিতরে `break` ব্যবহার করা যায়?

না।

Early exit দরকার হলে `for`, `for...of` ইত্যাদি ব্যবহার করা যায়।

58. Mental Model

পুরো concept এক লাইনে:

```
break
→ STOP

continue
→ SKIP

break label
→ STOP THAT LOOP

continue label
→ SKIP TO NEXT ITERATION OF THAT LOOP

return
→ EXIT FUNCTION
```

59. Real-World Mental Model

ধরো তুমি একটা দোকানে product খুঁজছো।

break

Product পেয়ে গেছি → আর search করার দরকার নেই।

```
break;
```

continue

এই product valid না → এটা বাদ দাও, পরেরটা দেখো।

```
continue;
```

break outerLoop

এই পুরো search process-ই বন্ধ করো।

```
break outerLoop;
```

`return`

পুরো *function*-এর কাজ শেষ।

```
return;
```

60. Quick Revision

Concept	Meaning
<code>break</code>	Loop/switch stop
<code>continue</code>	Current iteration skip
<code>break</code> in nested loop	Nearest loop stop
<code>break label</code>	Specific labeled loop stop
<code>continue label</code>	Specific labeled loop-এর next iteration
<code>switch break</code>	Fall-through prevent
<code>return</code>	Function exit
<code>forEach + break</code>	Directly possible না
<code>while + continue</code>	Infinite loop-এর risk
Labels	Rarely use; complex nested flow control

61. Golden Rules ★

Rule 1

`break` মানে:

পুরো *loop* বন্ধ।

Rule 2

`continue` মানে:

শুধু এই *iteration* বাদ।

Rule 3

Nested loop-এ:

`break` / `continue` nearest loop-কে affect করে।

Rule 4

Outer loop control করতে চাইলে:

```
break outerLoop;
```

অথবা:

```
continue outerLoop;
```

Rule 5

`while` loop-এ `continue` দিলে update logic carefully check করবে।

Rule 6

`switch` -এ `break` না দিলে fall-through হতে পারে।

Rule 7

`return` আর `break` এক জিনিস না।

```
break → loop  
return → function
```

Rule 8

Labels powerful হলেও unnecessarily ব্যবহার করবে না।

62. Section 9 — Final Mastery Checklist

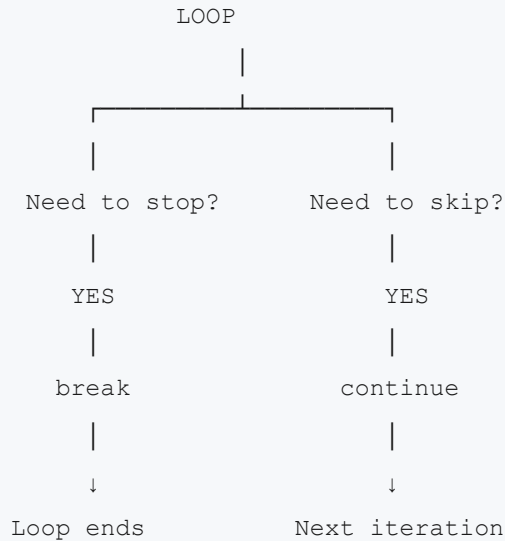
এখন তোমার **Loops & Iteration** section complete! নিচের সবগুলো confidently explain করতে পারলে sectionটা ভালোভাবে done:

- ☐ `for` loop
- ☐ `while` loop
- ☐ `do...while`
- ☐ loop initialization
- ☐ loop condition
- ☐ loop update
- ☐ infinite loop
- ☐ nested loops
- ☐ `for...of`
- ☐ iterable
- ☐ arrays with `for...of`
- ☐ strings with `for...of`
- ☐ Set with `for...of`
- ☐ Map with `for...of`
- ☐ `for...in`
- ☐ object keys
- ☐ `Object.keys()`
- ☐ `Object.values()`
- ☐ `Object.entries()`
- ☐ own vs inherited properties
- ☐ `Object.hasOwn()`
- ☐ `break`
- ☐ `continue`
- ☐ nested loop control
- ☐ labeled statements
- ☐ labeled `break`
- ☐ labeled `continue`
- ☐ `break` in `switch`
- ☐ fall-through
- ☐ `break` vs `continue`
- ☐ `break` vs `return`
- ☐ `continue` + `while` infinite-loop issue
- ☐ practical search patterns
- ☐ matrix/nested-loop patterns



Final Mental Model

Loops-এর পুরো control flow এভাবে মনে রাখো:



Nested loop হলো:

```
break
↓
nearest loop

break outerLoop
↓
specific outer loop
```

আর function-এর ক্ষেত্রে:

```
return
↓
entire function ends
```

এই তিনটা শব্দ — **break**, **continue**, **return** — ভালোভাবে আলাদা করতে পারা **loop control** বোঝার জন্য খুবই **important**!



Section 9 Complete

তোমার **Loops & Iteration** section এখন পুরোপুরি complete:

SECTION 9 – LOOPS & ITERATION

- └─  Doc 9.1
 - | for / while / do...while
- └─  Doc 9.2
 - | for...of / for...in
- └─  Doc 9.3
 - | break / continue / labeled statements

এর পরের JavaScript section-এ যাওয়ার জন্য তোমার loop fundamentals এখন solid হওয়া উচিত।